

Report Exercise 4
Advanced Computer Architectures

()

Michiel Vandromme & Joffrey Van Cauwenberghe

March 18, 2024



Class: Master Software Engineering

1 Introduction

In the first part of this lab, we investigated the impact of coalesced memory access. Coalesced memory access means that the result of multiple memory accesses can be obtained by just one memory access. The task was to create two kernels that compute the greyscale values of an image: one kernel with coalesced memory access and one kernel without coalesced memory access.

In the second part of the lab, we delved into the various forms of memory that can occur in GPU programming, namely Global, Shared and Constant Memory. For this, we developed three kernels (one using each of the memory types) to multiply two matrices with each other.

2 Implementation

2.1 Coalesced memory access

For the kernel that does not use Coalesced memory access, the pixels in the image array are sorted according to RGBRGB...RGB. Each thread reads three consecutive pixels each time and then calculates the greyscale value.

In the kernel that does use Coalesced memory access, the pixels in the image array are sorted as RR...RGG...GBB...B. This ensures that when Thread 1 fetches an R value, it will also automatically fetch other R values that can be used by other threads, without having had to fetch this value separately. This therefore allows a thread to compute its greyscale value without the need of accessing memory. This is an optimisation of threads within warps.



Since the first version retrieves data in groups of three, which is not an integer divisor of 32 (warp size), we experience less of the benefits of coalesced memory access here, which will result in a little more overhead.

2.2 Global, shared, and constant memory

In the first kernel for multiplying two matrices, we only used global memory. That is the memory accessible to all threads running on the GPU. In this implementation, each thread is responsible for computing one element in the matrix. Each thread will always request a row from A and a column from B from global memory to compute the element. This ensures that each row in A and each column in B, is read n times where n is the dimension of the matrix.

In the second kernel, we used shared memory in addition to global memory. Shared memory is on-chip memory that is shared between threads of a block. The idea here is to divide the array into blocks/tiles that can be loaded into the shared memory. This ensures that an element can only do part of the calculations. When all threads within a

block have finished their calculations, the next block is loaded into the shared memory so that the rest of the calculations can be performed.

The functionality of the kernel using constant memory is the same as in the first kernel, with the difference being that global memory is loaded into constant memory.

3 Evaluation & Conclusion

Figure 1 shows the difference in computation time to perform the image-greyscale task. As we could expect, the uncoalesced version is a bit slower. Attached are also the results of some additional experiments with different grid size/block size.

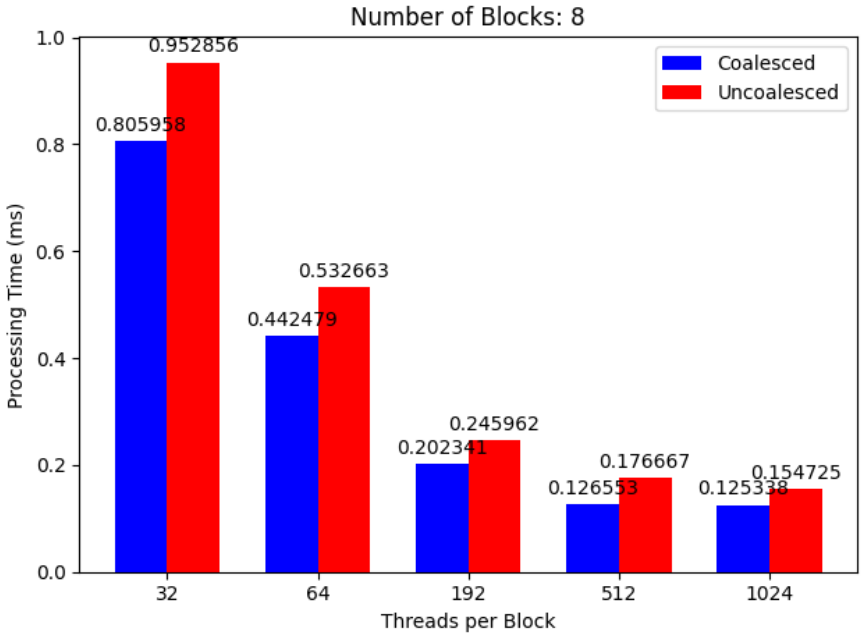


Figure 1: Processing Time Coalesced vs Uncoalesced Memory Access

The figures below 2 3 show the difference in computation time to perform the matrix-multiplication task. Again, the results are as expected when we add a diagram of the hardware architecture. Memory closer to the GPU (based on the architecture scheme in the slides of the lab) has higher performance and provides more efficient reads. Unfortunately, in the experiment with the largest matrix, we could not use constant memory because constant memory on our PC is only allowed to occupy 8kB (which is too small for these matrices).

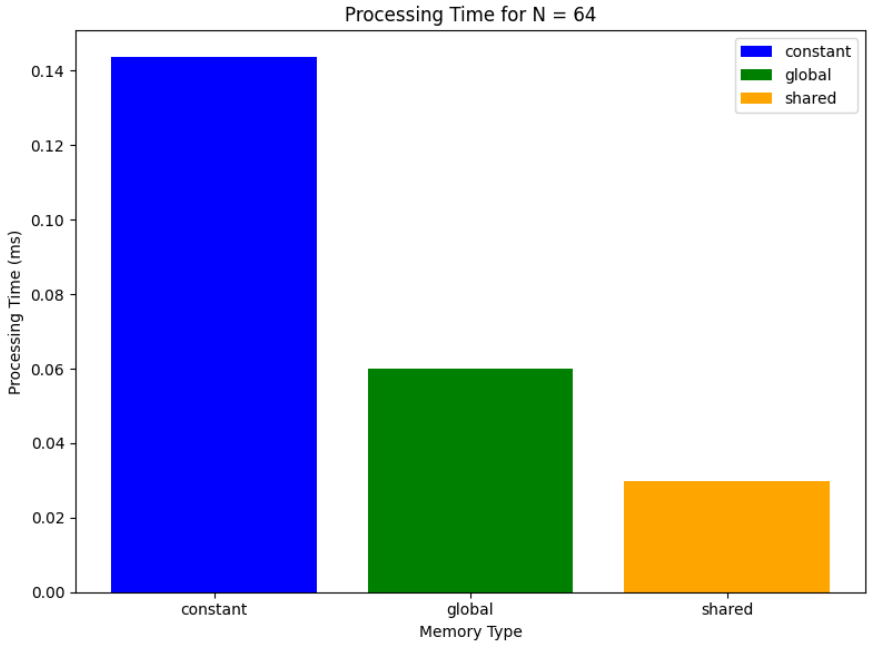


Figure 2: Processing Time NxN Matrix Multiplication with different memory types

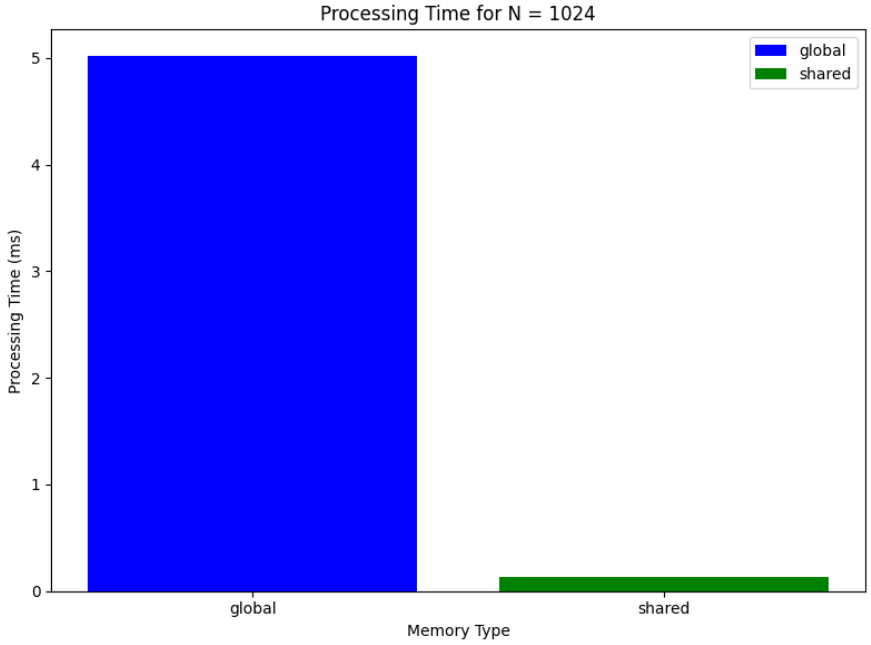


Figure 3: Processing Time Bigger NxN Matrix Multiplication with different memory types